Reporte de Práctica Guiada: Actividad 4 – Implementación del núcleo de negocio TechStore

Nombres (EQUIPO 3): Cristopher Alan Pulgarin Santa Cruz Francisco Sebastián Hernández Terán Jesús Orlando Moreno Chávez	**Matrícula**: 2105894 2115798 2108300
Nombre del curso: Componentes de Lógica de Negocios	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Módulo Netacad: 4	**Actividad**: Implementación del núcleo de negocio TechStore
Fecha: 26/10/2026	

Bibliografía

1. Domain Model, Entidades y Domain Events (Domain-Driven Design)
   - Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley Professional.
   - Vernon, V. (2013). *Implementing domain-driven design*. Addison-Wesley.
2. Patrón Specification y reglas de negocio encapsuladas
   - Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.
3. Aislamiento del núcleo de dominio (Arquitectura Limpia)
   - Martin, R. C. (2017). *Arquitectura limpia: Guía para especialistas en la estructura y el diseño de software*. Anaya Multimedia.
4. API REST, FastAPI y manejo de excepciones en la capa de infraestructura
   - Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine].
   - Ramírez, S. (s. f.). *FastAPI*. https://fastapi.tiangolo.com/
   - Encode. (s. f.). *Uvicorn*. https://www.uvicorn.org/
5. Pruebas automatizadas y aritmética monetaria en Python
   - Okken, B. (2022). *Python testing with pytest* (2.ª ed.). The Pragmatic Bookshelf.
   - Python Software Foundation. (s. f.). *decimal: Decimal fixed-point and floating-point arithmetic*. https://docs.python.org/3/library/decimal.html

Índice

1. Introducción y alcance 3
Componentes implementados 3
Reglas de negocio aplicadas 3
2. Evidencia de Carga de Datos (Fase 1) 4
3. Bitácora de Escenarios (Fase 2) 4
4. Pruebas de API REST (Fase 3) 7
Operación Exitosa (HTTP 200) 7
Operación Rechazada (HTTP 4xx) 8
Resumen de respuestas HTTP 9
5. Pruebas Automatizadas Unitarias (Fase 4) 10
Catálogo de pruebas 11
6. Diagrama de Estados y Domain Events 12
7. Reflexión Breve 12
8. Respuestas a Preguntas del Manual 13
9. Conclusiones 14
Cumplimiento de la lista de verificación 14

1. Introducción y alcance

Este reporte documenta la implementación del núcleo de negocio de TechStore, una tienda ficticia que administra pedidos. El sistema lee clientes, productos y pedidos desde archivos CSV, los convierte en objetos del dominio, aplica reglas de descuento, controla el ciclo de vida de cada pedido mediante una máquina de estados y expone sus operaciones mediante una API REST. Todo se trabaja con objetos en memoria, por lo que no se usa base de datos.

El diseño sigue el principio central de la unidad: las reglas de negocio viven en el dominio y no en la interfaz. El controlador HTTP solo busca el pedido, invoca el método correspondiente de Order y traduce las excepciones del dominio a códigos HTTP. Gracias a esto, la misma lógica puede probarse sin servidor y reutilizarse desde otras interfaces.

Componentes implementados

Archivo	Responsabilidad
app/domain.py	Value Objects Money y OrderItem; entidades Customer y Order (Aggregate Root); máquina de estados, DomainEvent y jerarquía de excepciones de dominio.
app/specifications.py	Specification Pattern con composición AND (&), OR (|) y NOT (~); reglas MinimumAmountSpec, VIPCustomerSpec, MinimumUnitsSpec y ActivePromotionSpec.
app/api.py	API REST con FastAPI: busca el pedido, invoca el dominio y traduce excepciones a 404, 409 y 422.
cargar_csv.py	Carga los cinco CSV y construye los 16 pedidos en estado DRAFT.
generar_bitacora.py	Ejecuta los escenarios E01 a E15 y muestra el resultado de cada operación.
tests/	test_domain.py y test_api.py, más test_reglas_adicionales.py: 30 pruebas con pytest.

Reglas de negocio aplicadas

Regla	Condición
Límite de pedido	Máximo 20 unidades por pedido. Se cuenta la suma de unidades, no el número de renglones.
Descuento del 10 %	Monto mayor a 500 (estricto), cliente VIP, al menos 4 unidades y sin promoción activa.
Descuento del 5 %	Monto mayor a 5000 y sin promoción activa. Si se cumplen ambas reglas, se aplica primero el 10 %.
Confirmar	Solo desde DRAFT, con al menos un artículo y con cliente activo.
Enviar	Solo desde CONFIRMED y con número de guía no vacío.
Cancelar	Solo desde CONFIRMED y con motivo no vacío.
Entregar / Devolver	Solo desde SHIPPED.

2. Evidencia de Carga de Datos (Fase 1)

El sistema carga correctamente el catálogo base convirtiendo los registros CSV en entidades del dominio. Todos los pedidos inician en su estado base.

Al ejecutar cargar_csv.py se construyeron los 16 pedidos del paquete y todos aparecen en estado DRAFT. Como verificación, O001 muestra 4 unidades y un total de 2100.00 MXN, valor que coincide con el cálculo manual. También se observan los casos frontera que usarán los escenarios: O007 sin artículos (0 unidades), O008 con 20 unidades exactas, O009 con 19 unidades y O010 con un total de exactamente 500.00.

```
(.venv) PS C:\Users\crist\Desktop\techstore-u4> python cargar_csv.py
O001 DRAFT 4 2100.00
O002 DRAFT 2 1050.00
O003 DRAFT 4 2100.00
O004 DRAFT 1 850.00
O005 DRAFT 4 2100.00
O006 DRAFT 1 18000.00
O007 DRAFT 0 0
O008 DRAFT 20 2400.00
O009 DRAFT 19 2280.00
O010 DRAFT 2 500.00
O011 DRAFT 3 1500.00
O012 DRAFT 1 850.00
O013 DRAFT 1 600.00
O014 DRAFT 1 600.00
O015 DRAFT 1 450.00
O016 DRAFT 1 2100.00
(.venv) PS C:\Users\crist\Desktop\techstore-u4>
```

Figura 1. Carga de los 16 pedidos desde los CSV, todos en estado DRAFT.

3. Bitácora de Escenarios (Fase 2)

Ejecución aislada de las operaciones evaluando los límites matemáticos y la máquina de estados.

Caso	Estado Inicial	Operación (Código)	Resultado y Eventos	Explicación de la Regla
E01	DRAFT	regla_10.is_satisfied_by()	Cumple (True) Evento: Ninguno	El pedido tiene >$500, >=4 unidades, es VIP y no hay promo activa.
E02	DRAFT	regla_5.is_satisfied_by()	No cumple (False) Evento: Ninguno	El cliente no es VIP y su monto de $1050 no llega al umbral de $5000.
E03	DRAFT	order.confirm()	Fallo: InvalidOrderData Evento: Ninguno	El dominio rechaza la confirmación porque el atributo active del cliente es falso.
E04	DRAFT	reglas.is_satisfied_by()	No cumple (False) Evento: Ninguno	La existencia de una promoción activa (has_promotion=True) invalida los descuentos.
E05	DRAFT	regla_5.is_satisfied_by()	Cumple (True) Evento: Ninguno	El monto de $18,000 supera el umbral estricto de $5000 sin promoción activa.
E06	DRAFT	order.confirm()	Fallo: InvalidOrderData Evento: Ninguno	La regla if not self.items impide confirmar un pedido sin artículos.

Caso	Estado Inicial	Operación (Código)	Resultado y Eventos	Explicación de la Regla
E07	DRAFT	order.confirm()	Éxito (CONFIRMED) Evento: OrderConfirmed	El pedido de 20 unidades exactas es válido ya que no supera el límite máximo.
E08	DRAFT	order.add_item()	Fallo: OrderItemLimitExceeded Evento: Ninguno	Intentar añadir artículos a un pedido de 19 unidades que resulta en >20 unidades bloquea la adición.
E09	DRAFT	regla_10.is_satisfied_by()	No cumple (False) Evento: Ninguno	El monto exacto de $500.00 falla en la evaluación > 500 (umbral estricto).
E10	DRAFT	regla_10.is_satisfied_by()	No cumple (False) Evento: Ninguno	El pedido cuenta con 3 unidades, por lo que no alcanza el mínimo de 4 unidades requerido.
E11	CONFIRMED	order.ship("")	Fallo: InvalidOrderData Evento: Ninguno	Se rechaza la transición a enviado por enviar un string vacío como número de guía.
E12	CONFIRMED	order.confirm()	Fallo: InvalidOrderTransition Evento: Ninguno	La máquina de estados impide volver a confirmar un pedido que ya salió de DRAFT.
E13	SHIPPED	order.cancel("Motivo")	Fallo: InvalidOrderTransition Evento: Ninguno	No existe una transición válida entre SHIPPED y CANCELLED.
E14	SHIPPED	order.return_order()	Éxito (RETURNED) Evento: OrderReturned	Transición válida que respeta el flujo de vida del pedido.
E15	SHIPPED	order.deliver()	Éxito (DELIVERED) Evento: OrderDelivered	Camino feliz completo; el pedido culmina su ciclo de forma satisfactoria.

Cada fila registra el caso, el estado del pedido antes de la operación, la operación ejecutada, el resultado real con el evento emitido y la regla que lo explica. Los resultados registrados muestran que las operaciones inválidas lanzan una excepción de dominio y no emiten evento, y las válidas cambian el estado y emiten el evento correspondiente.

Nota sobre la columna «Estado Inicial»: el script generar_bitacora.py imprime el estado con el que se carga el pedido (DRAFT). En los escenarios E11 a E15, la tabla indica el estado en que se encuentra el pedido justo antes de ejecutar la operación evaluada, después de las transiciones previas necesarias (por ejemplo, confirmar y enviar).

```
(.venv) PS C:\Users\crist\Desktop\techstore-u4> & c:\Users\crist\Desktop\techstore-u4\.venv\Scripts\python.exe c:/Users/crist/Desktop
/techstore-u4/generar_bitacora.py
Estado inicial: DRAFT, Total: 2100.00, Unidades: 4
¿Cumple el descuento del 10%?: True
Éxito. Estado actual: DRAFT
(.venv) PS C:\Users\crist\Desktop\techstore-u4> & c:\Users\crist\Desktop\techstore-u4\.venv\Scripts\python.exe c:/Users/crist/Desktop
/techstore-u4/generar_bitacora.py
Estado inicial: DRAFT, Total: 1050.00, Unidades: 2
¿Cumple 10%?: False
¿Cumple 5%?: False
Éxito. Estado actual: DRAFT
(.venv) PS C:\Users\crist\Desktop\techstore-u4> & c:\Users\crist\Desktop\techstore-u4\.venv\Scripts\python.exe c:/Users/crist/Desktop
/techstore-u4/generar_bitacora.py
Estado inicial: DRAFT, Total: 2100.00, Unidades: 4
Fallo: InvalidOrderData - Requiere artículos y cliente activo
(.venv) PS C:\Users\crist\Desktop\techstore-u4> & c:\Users\crist\Desktop\techstore-u4\.venv\Scripts\python.exe c:/Users/crist/Desktop
/techstore-u4/generar_bitacora.py
Estado inicial: DRAFT, Total: 2100.00, Unidades: 4
¿Cumple 10%?: False
¿Cumple 5%?: False
Éxito. Estado actual: DRAFT
(.venv) PS C:\Users\crist\Desktop\techstore-u4> & c:\Users\crist\Desktop\techstore-u4\.venv\Scripts\python.exe c:/Users/crist/Desktop
/techstore-u4/generar_bitacora.py
Estado inicial: DRAFT, Total: 18000.00, Unidades: 1
¿Cumple 5%?: True
Éxito. Estado actual: DRAFT
(.venv) PS C:\Users\crist\Desktop\techstore-u4> & c:\Users\crist\Desktop\techstore-u4\.venv\Scripts\python.exe c:/Users/crist/Desktop
/techstore-u4/generar_bitacora.py
Estado inicial: DRAFT, Total: 0, Unidades: 0
Fallo: InvalidOrderData - Requiere artículos y cliente activo
(.venv) PS C:\Users\crist\Desktop\techstore-u4> & c:\Users\crist\Desktop\techstore-u4\.venv\Scripts\python.exe c:/Users/crist/Desktop
/techstore-u4/generar_bitacora.py
Estado inicial: DRAFT, Total: 2400.00, Unidades: 20
Éxito. Estado actual: CONFIRMED
Último evento emitido: OrderConfirmed
```

Figura 2. Salida de generar_bitacora.py para los escenarios E01 a E07.

```
(.venv) PS C:\Users\crist\Desktop\techstore-u4> & c:\Users\crist\Desktop\techstore-u4\.venv\Scripts\python.exe c:/Users/crist/Desktop
/techstore-u4/generar_bitacora.py
Estado inicial: DRAFT, Total: 2280.00, Unidades: 19
Fallo: OrderItemLimitExceeded - Máximo 20 unidades por pedido
(.venv) PS C:\Users\crist\Desktop\techstore-u4> & c:\Users\crist\Desktop\techstore-u4\.venv\Scripts\python.exe c:/Users/crist/Desktop
/techstore-u4/generar_bitacora.py
Estado inicial: DRAFT, Total: 500.00, Unidades: 2
¿Cumple 10%?: False
Éxito. Estado actual: DRAFT
(.venv) PS C:\Users\crist\Desktop\techstore-u4> & c:\Users\crist\Desktop\techstore-u4\.venv\Scripts\python.exe c:/Users/crist/Desktop
/techstore-u4/generar_bitacora.py
Estado inicial: DRAFT, Total: 1500.00, Unidades: 3
¿Cumple 10%?: False
Éxito. Estado actual: DRAFT
(.venv) PS C:\Users\crist\Desktop\techstore-u4> & c:\Users\crist\Desktop\techstore-u4\.venv\Scripts\python.exe c:/Users/crist/Desktop
/techstore-u4/generar_bitacora.py
Estado inicial: DRAFT, Total: 850.00, Unidades: 1
Fallo: InvalidOrderData - Requiere número de guía
(.venv) PS C:\Users\crist\Desktop\techstore-u4> & c:\Users\crist\Desktop\techstore-u4\.venv\Scripts\python.exe c:/Users/crist/Desktop
/techstore-u4/generar_bitacora.py
Estado inicial: DRAFT, Total: 600.00, Unidades: 1
Fallo: InvalidOrderTransition - Solo DRAFT puede ser confirmado
(.venv) PS C:\Users\crist\Desktop\techstore-u4> & c:\Users\crist\Desktop\techstore-u4\.venv\Scripts\python.exe c:/Users/crist/Desktop
/techstore-u4/generar_bitacora.py
Estado inicial: DRAFT, Total: 600.00, Unidades: 1
Fallo: InvalidOrderTransition - Solo CONFIRMED puede ser cancelado
(.venv) PS C:\Users\crist\Desktop\techstore-u4> & c:\Users\crist\Desktop\techstore-u4\.venv\Scripts\python.exe c:/Users/crist/Desktop
/techstore-u4/generar_bitacora.py
Estado inicial: DRAFT, Total: 450.00, Unidades: 1
Éxito. Estado actual: RETURNED
Último evento emitido: OrderReturned
(.venv) PS C:\Users\crist\Desktop\techstore-u4> & c:\Users\crist\Desktop\techstore-u4\.venv\Scripts\python.exe c:/Users/crist/Desktop
/techstore-u4/generar_bitacora.py
Estado inicial: DRAFT, Total: 2100.00, Unidades: 1
Éxito. Estado actual: DELIVERED
```

Figura 3. Salida de generar_bitacora.py para los escenarios E08 a E15.

4. Pruebas de API REST (Fase 3)

La capa de infraestructura HTTP recibe las excepciones del dominio y las traduce correctamente a códigos semánticos.

Operación Exitosa (HTTP 200)

Se confirmó el pedido O001 con POST /orders/O001/confirm. La API respondió 200 y devolvió el nuevo estado (CONFIRMED) junto con el evento OrderConfirmed registrado por el dominio.

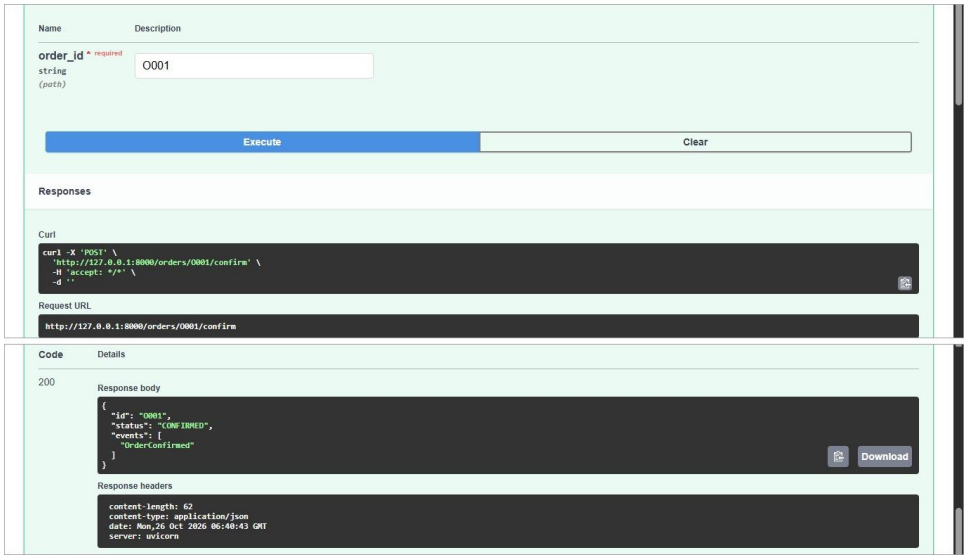


Figura 4. POST /orders/O001/confirm: respuesta 200 con estado CONFIRMED y evento OrderConfirmed.

Operación Rechazada (HTTP 4xx)

Se probaron tres rechazos. En todos, el controlador no valida reglas: el dominio lanza la excepción y la API la traduce a un código HTTP semántico.

404 Not Found. El pedido O999 no existe en el diccionario de pedidos; la API responde «No existe».

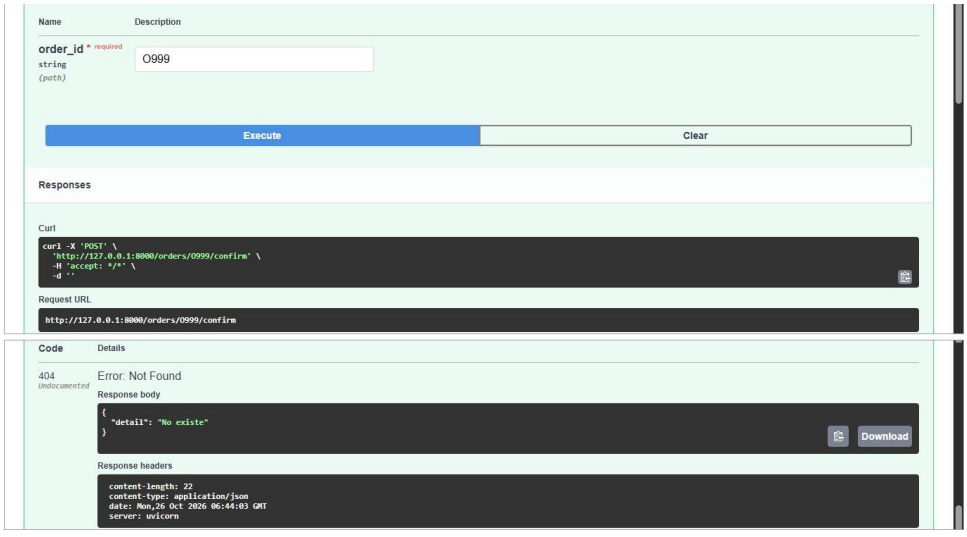


Figura 5. POST /orders/O999/confirm: respuesta 404.

409 Conflict. O013 se confirmó una vez y se intentó confirmar de nuevo. El dominio lanza InvalidOrderTransition («Solo DRAFT puede ser confirmado») y la API responde 409.

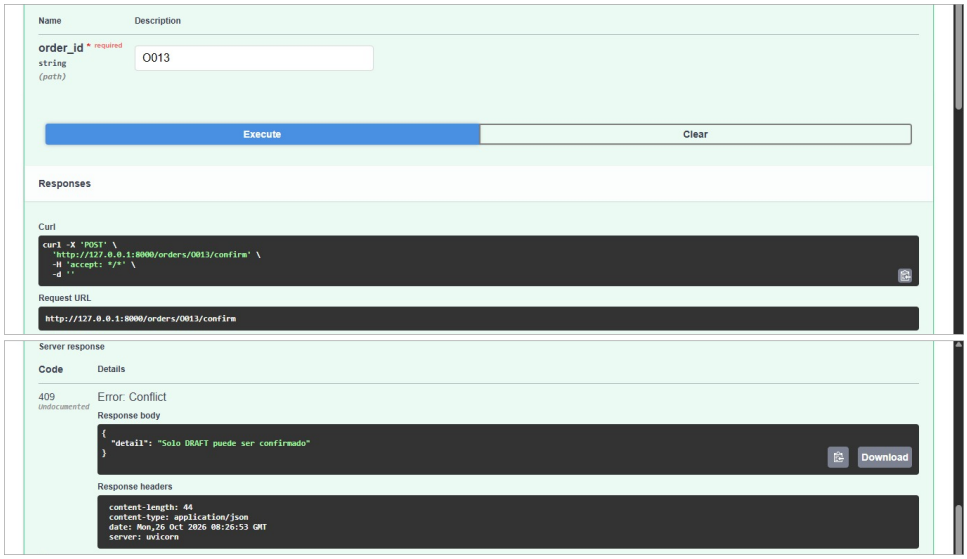


Figura 6. Segunda confirmación de O013: respuesta 409.

422 Unprocessable Content. O007 no tiene artículos. El dominio lanza InvalidOrderData («Requiere artículos y cliente activo») y la API responde 422.

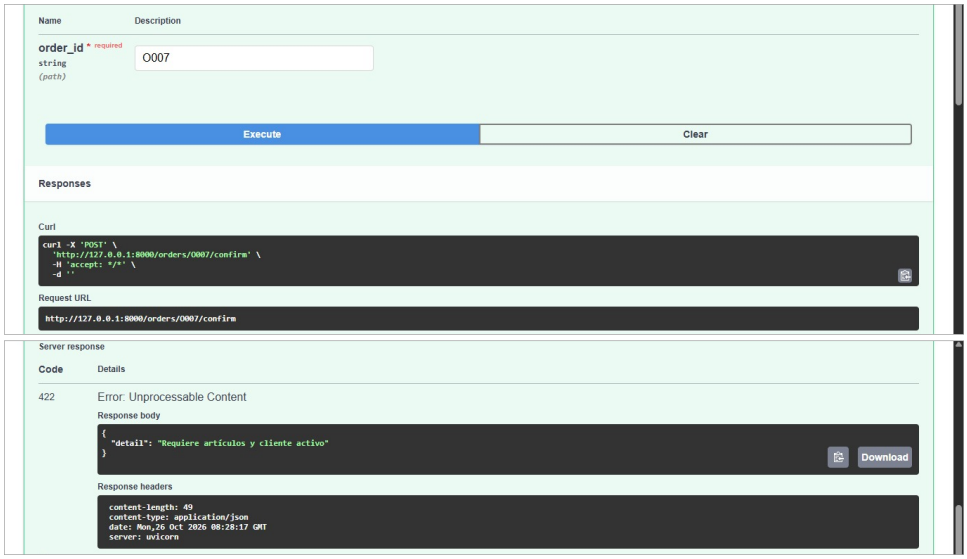


Figura 7. POST /orders/O007/confirm: respuesta 422.

Resumen de respuestas HTTP

Operación	Pedido	Código	Excepción de dominio	Mensaje
POST /orders/{id}/confirm	O001	200	Ninguna	Estado CONFIRMED; evento OrderConfirmed
POST /orders/{id}/confirm	O999	404	(no existe el pedido)	No existe
POST /orders/{id}/confirm	O013 (2.ª vez)	409	InvalidOrderTransition	Solo DRAFT puede ser confirmado
POST /orders/{id}/confirm	O007	422	InvalidOrderData	Requiere artículos y cliente activo

5. Pruebas Automatizadas Unitarias (Fase 4)

Las invariantes, validaciones matemáticas y endpoints están cubiertos por tests aislados.

Se ejecutaron 30 pruebas con pytest y todas pasaron. Cada prueba sigue el esquema preparar datos, ejecutar la acción y comprobar el resultado. Cuando una operación es inválida, además de comprobar la excepción se verifica que el estado no cambie y que no se agregue ningún evento.

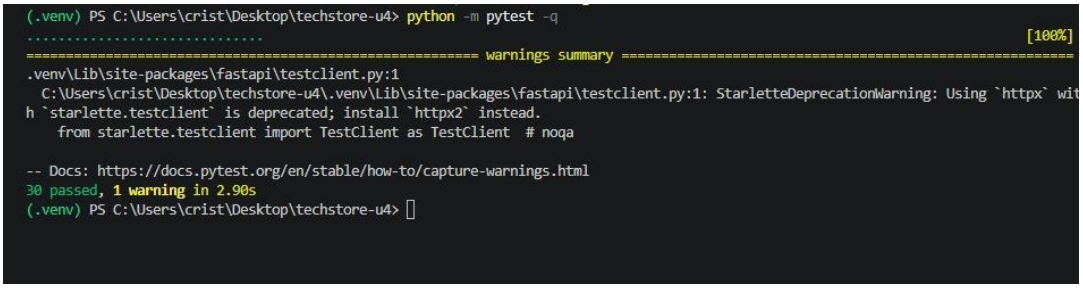


Figura 8. Resultado final: python -m pytest -q, 30 passed.

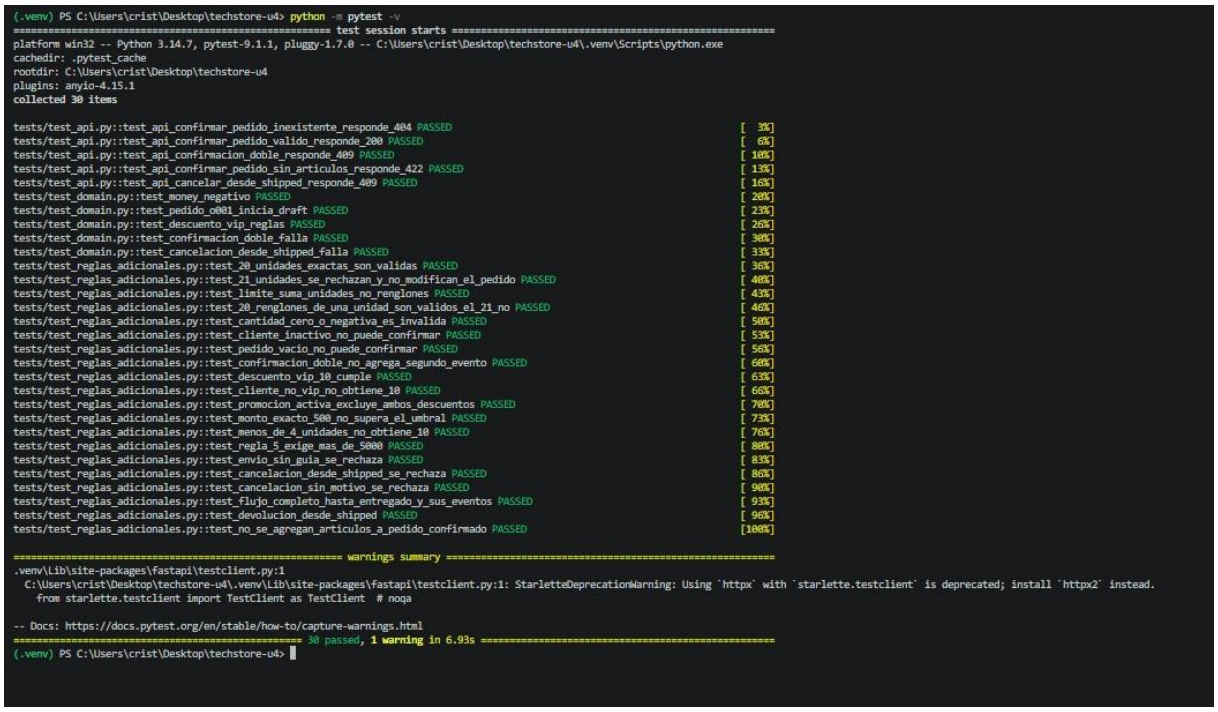


Figura 9. Detalle de las 30 pruebas: python -m pytest -v.

La advertencia StarletteDeprecationWarning proviene de la dependencia fastapi.testclient/httpx; no afecta el resultado de las pruebas.

Catálogo de pruebas

#	Archivo	Prueba	Qué comprueba
1	test_api	confirmar_pedido_inexistente_responde_404	Un ID inexistente responde 404.
2	test_api	confirmar_pedido_valido_responde_200	Un pedido válido se confirma y responde 200.
3	test_api	confirmacion_doble_responde_409	Confirmar dos veces responde 409.
4	test_api	confirmar_pedido_sin_articulos_responde_422	Un pedido sin artículos responde 422.
5	test_api	cancelar_desde_shipped_responde_409	Cancelar un pedido enviado responde 409.
6	test_domain	money_negativo	Money rechaza montos negativos.
7	test_domain	pedido_o001_inicia_draft	O001 inicia en DRAFT con 4 unidades y 2100.00.
8	test_domain	descuento_vip_reglas	Reglas de descuento VIP con Specification.
9	test_domain	confirmacion_doble_falla	La segunda confirmación lanza InvalidOrderTransition.
10	test_domain	cancelacion_desde_shipped_falla	No se puede cancelar desde SHIPPED.
11	test_reglas_adicionales	20_unidades_exactas_son_validas	Un pedido de 20 unidades es válido.
12	test_reglas_adicionales	21_unidades_se_rechazan_y_no_modifican_el_pedido	La unidad 21 se rechaza sin alterar el pedido.
13	test_reglas_adicionales	limite_suma_unidades_no_renglones	Con 19 unidades, agregar 2 se rechaza: el límite suma unidades.
14	test_reglas_adicionales	20_renglones_de_una_unidad_son_validos_el_21_no	Veinte renglones de 1 unidad son válidos; el 21 no.
15	test_reglas_adicionales	cantidad_cero_o_negativa_es_invalida	OrderItem rechaza cantidades 0 y negativas.
16	test_reglas_adicionales	cliente_inactivo_no_puede_confirmar	Cliente inactivo: falla y el estado sigue en DRAFT sin eventos.
17	test_reglas_adicionales	pedido_vacio_no_puede_confirmar	Pedido vacío: falla sin cambiar estado ni eventos.
18	test_reglas_adicionales	confirmacion_doble_no_agrega_segundo_evento	Solo existe un OrderConfirmed tras la doble confirmación.
19	test_reglas_adicionales	descuento_vip_10_cumple	Pedido VIP de 4 unidades y 800 cumple el 10 %.
20	test_reglas_adicionales	cliente_no_vip_no_obtiene_10	Un cliente no VIP no obtiene el 10 %.
21	test_reglas_adicionales	promocion_activa_excluye_ambos_descuentos	Con promoción activa no aplican el 10 % ni el 5 %.
22	test_reglas_adicionales	monto_exacto_500_no_supera_el_umbral	Un total de exactamente 500.00 no supera el umbral.
23	test_reglas_adicionales	menos_de_4_unidades_no_obtiene_10	Con 3 unidades no se obtiene el 10 %.
24	test_reglas_adicionales	regla_5_exige_mas_de_5000	Con 5000 exactos no cumple; con 5001 sí.
25	test_reglas_adicionales	envio_sin_guia_se_rechaza	Enviar con guía vacía falla; el pedido sigue CONFIRMED.
26	test_reglas_adicionales	cancelacion_desde_shipped_se_rechaza	Cancelar desde SHIPPED falla y no emite OrderCancelled.
27	test_reglas_adicionales	cancelacion_sin_motivo_se_rechaza	Cancelar sin motivo falla; el estado no cambia.
28	test_reglas_adicionales	flujo_completo_hasta_entregado_y_sus_eventos	Confirmar, enviar y entregar emite los tres eventos en orden.
29	test_reglas_adicionales	devolucion_desde_shipped	Desde SHIPPED se puede devolver; emite OrderReturned.
30	test_reglas_adicionales	no_se_agregan_articulos_a_pedido_confirmado	Solo DRAFT admite agregar artículos.

6. Diagrama de Estados y Domain Events

El flujo implementado restringe las modificaciones libres y emite eventos en pasado solo tras validar la operación:

- DRAFT → (confirm) → CONFIRMED [Emite: OrderConfirmed]
    - Desde CONFIRMED → (cancel) → CANCELLED [Emite: OrderCancelled]
    - Desde CONFIRMED → (ship) → SHIPPED [Emite: OrderShipped]
        - Desde SHIPPED → (return) → RETURNED [Emite: OrderReturned]
        - Desde SHIPPED → (deliver) → DELIVERED [Emite: OrderDelivered]

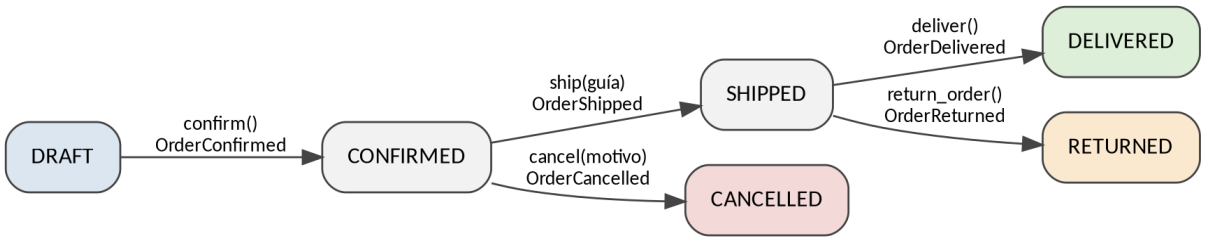


Figura 10. Máquina de estados de Order y eventos emitidos en cada transición.

Regla común de todas las transiciones: primero se valida, después se cambia el estado y, solo al final, se registra el evento. Así, una operación fallida nunca anuncia un hecho que no ocurrió.

Evento	Método	Condiciones	Transición
OrderConfirmed	confirm()	Estado DRAFT, al menos un artículo y cliente activo	DRAFT → CONFIRMED
OrderShipped	ship(guía)	Estado CONFIRMED y número de guía no vacío	CONFIRMED → SHIPPED
OrderCancelled	cancel(motivo)	Estado CONFIRMED y motivo no vacío	CONFIRMED → CANCELLED
OrderDelivered	deliver()	Estado SHIPPED	SHIPPED → DELIVERED
OrderReturned	return_order()	Estado SHIPPED	SHIPPED → RETURNED

Los eventos son objetos inmutables (DomainEvent) con el nombre del evento, el identificador del pedido y la marca de tiempo en que ocurrieron. Se nombran en pasado porque comunican hechos ya sucedidos, y el Order los acumula sin conocer a sus posibles suscriptores.

7. Reflexión Breve

Error 1: estado compartido entre pruebas. Al principio, el método de limpieza de arrays mutaba el objeto base, de modo que un pedido modificado en una prueba llegaba alterado a las siguientes y los resultados dependían del orden de ejecución. **Solución:** recargar los fixtures desde los CSV para cada ejecución, de modo que cada prueba y cada escenario parte de pedidos nuevos en DRAFT. **Aprendizaje:** una prueba aislada no debe heredar el estado de otra.

Error 2: módulo equivocado al iniciar el servidor. Uvicorn falló al arrancar por una importación incorrecta (app.app:app en lugar de app.api:app). **Solución:** entender que Uvicorn recibe la ruta del módulo seguida del nombre del objeto FastAPI, es decir, uvicorn app.api:app --reload. **Aprendizaje:** la capa de infraestructura se configura por separado y no afecta al dominio.

Error 3: «Failed to fetch» en Swagger. Al confirmar dos veces O013, Swagger mostró «Failed to fetch» y el código «Undocumented» en lugar del 409 esperado. No era un error de la API: el servidor Uvicorn se había detenido y la petición nunca llegó. **Solución:** volver a iniciar el servidor, recargar /docs y repetir la prueba, con lo que apareció el 409 correcto. **Aprendizaje:** conviene revisar la terminal del servidor antes de culpar al código.

Aprendizaje general. Aislar la lógica de negocio en el centro asegura que ni la consola ni una API REST puedan corromper un pedido, centralizando el control. También se comprobó que la primera versión del conjunto de pruebas tenía 10 casos y dejaba sin cubrir los valores frontera (20 y 21 unidades), el cliente inactivo, el envío sin guía y la promoción activa. Al ampliarlo a 30 pruebas se cubrió cada regla en su caso válido y en su caso inválido.

8. Respuestas a Preguntas del Manual

¿Por qué se usa Decimal en lugar de float para precios?
Float utiliza representación binaria fraccional, lo que causa imprecisión en aritmética financiera (ej. 0.1 + 0.2 != 0.3). Decimal garantiza exactitud exacta en base 10 para operaciones monetarias.

¿Cuál es la diferencia entre contar renglones y sumar unidades?
Contar renglones (filas) mide la cantidad de SKUs distintos (ej. 1 línea de "Lápices"). Sumar unidades mide el inventario físico total (ej. la cantidad de esa línea es 50 lápices). El límite de 20 aplica al volumen físico, no a la variedad de catálogo.

¿Qué ventaja tiene encapsular una regla en Specification?
Evita saturar la entidad Order con condicionales masivos (if and or not). Las reglas se vuelven objetos combinables, reutilizables en múltiples servicios (ej. validaciones, reportes o queries) y fáciles de someter a pruebas unitarias aisladas.

¿Por qué los eventos se registran solo después de una transición válida?
Un "Domain Event" representa un hecho histórico inmutable (algo que ya pasó). Si se registrara antes de validar o si la validación falla, se comunicaría una falsedad al resto de los sistemas (ej. cobrar un pedido que internamente fue rechazado).

¿Qué parte del código podrías reutilizar si reemplazas FastAPI por otra interfaz?
Todo el núcleo del dominio. Los archivos domain.py y specifications.py son agnósticos a la infraestructura. Se pueden conectar a una interfaz gráfica de escritorio, un proceso Batch o Django sin modificar una sola regla matemática o estado.

9. Conclusiones

La práctica muestra que concentrar las reglas en el dominio hace que el sistema sea predecible: ni la consola ni la API pueden dejar un pedido en un estado inválido, porque toda operación pasa por los métodos de Order. Las 30 pruebas automatizadas y los 15 escenarios documentan este comportamiento, incluidos los valores frontera (20 y 21 unidades, monto exacto de 500 y de 5000).

La separación entre dominio y API se comprobó en la práctica: el controlador no contiene reglas y se limita a traducir las excepciones de dominio a los códigos 404, 409 y 422. Como el dominio puede ejecutarse sin servidor, es posible probarlo en memoria y reutilizarlo con otra interfaz.

10. Repositorio de Github

https://github.com/cristophersa08/TechStore-U4